Minh họa các bộ phân tích UPL hoạt động đúng

Dùng một chương trình nhỏ và các biến thể để đối chiếu kết quả mong đợi với kết quả thực tế. Đây là bằng chứng kiểm thử cho các trường hợp cụ thể, không phải chứng minh mọi chương trình UPL đều được xử lý đúng.

1. Chương trình kiểm thử

```
begin
    // Kiem tra thu tu uu tien
    int x=1+2*3;
    print(x);
end
```

Chạy từ thư mục gốc dự án bằng Cargo (cần Rust, Flex, Bison và trình biên dịch C):

```
cargo run --quiet --offline <<'UPL'
begin
    // Kiem tra thu tu uu tien
    int x=1+2*3;
    print(x);
end
UPL
```

2. Phân tích từng dòng và kết quả mong đợi

Lexer quét từ trái sang phải, bỏ qua khoảng trắng và comment, trả token khi parser cần. Parser ghép token theo văn phạm để tạo AST.

Dòng	Điều cần kiểm tra	Kết quả mong đợi
1. `begin`	Nhận từ khóa mở đầu.	Parser bắt đầu chương trình.
2. `// Kiem tra thu tu uu tien`	Bỏ qua comment.	Không tạo token hoặc nút AST cho comment.
3. `int x=1+2*3;`	Nhận khai báo và ưu tiên toán tử.	Tạo `Declaration` kiểu `int`, định danh `x`; biểu thức có phép `+` ở gốc và phép `*` ở nhánh phải.
4. `print(x);`	Nhận lời gọi `print` với biểu thức trong ngoặc.	Tạo `Print` chứa `Identifier(x)`.
5. `end`	Nhận kết thúc chương trình và cuối đầu vào.	Hoàn tất `Program`, không có lỗi, trả mã 0 và xuất AST JSON.

Token của dòng 3 (`ID` và `INTEGER` là tên viết gọn):

```
int  ID(x)  =  INTEGER(1)  +  INTEGER(2)  *  INTEGER(3)  ;
```

Cấu trúc AST mong đợi, lược bỏ thông tin dòng và cột:

```
Program
├── Declaration(int)
│   ├── Identifier(x)
│   └── Binary(+)
│       ├── Integer(1)
│       └── Binary(*)
│           ├── Integer(2)
│           └── Integer(3)
└── Print
    └── Identifier(x)
```

Cây này cho thấy parser hiểu `1+2*3` thành `1+(2*3)`. Kết quả của chương trình phân tích là AST; nó chưa tính giá trị `x` hoặc thực thi `print`.

3. Đối chiếu với kết quả thực tế

Giữ nguyên chương trình trên, chỉ thay dòng 3 trong từng trường hợp. Các trường hợp dưới đây đã được chạy bằng binary compiler hiện có; cấu trúc AST, thông báo lỗi và mã thoát đều khớp kết quả mong đợi.

Trường hợp / dòng 3	Kết quả mong đợi và đã quan sát	Chức năng được kiểm chứng
`int x=1+2*3;`	Mã 0, không báo lỗi; AST có `Program` chứa `Declaration`, `Print`; biểu thức có + ở gốc và * ở nhánh phải.	Khai báo, comment, câu lệnh `print`, ưu tiên * trước +.
`int x=(1+2)*3;`	Mã 0; biểu thức có * ở gốc và + ở nhánh trái.	Ngoặc thay đổi cách ghép biểu thức.
`int x=;`	Mã 1, không xuất AST; báo lỗi cú pháp ở dòng 3, cột 11 tại `;`.	Lexer nhận được token hợp lệ, parser phát hiện thiếu biểu thức sau =.
`int x=@;`	Mã 1, không xuất AST; báo lỗi từ vựng ở dòng 3, cột 11: ký tự @ không hợp lệ.	Lexer phát hiện ký tự ngoài tập token của UPL.

Các kết quả trên minh họa bộ phân tích nhận đúng chương trình hợp lệ, xây dựng đúng cấu trúc biểu thức và phân biệt lỗi từ vựng với lỗi cú pháp.

Phạm vi: ví dụ nhỏ này chưa kiểm chứng `if/else`, `do/while`, `for`, phục hồi sau nhiều lỗi hoặc mọi trường hợp biên. Chương trình hiện tại chưa kiểm tra ngữ nghĩa và chưa thực thi UPL.